Algorithms Foundations

Veronica Guerrini and Nadia Pisanti, Dipartimento di Informatica, University of Pisa, Pisa, Italy

Abstract

Algorithmics and data structures play a special role in the analysis of biological sequences, structures, and networks. Nowadays, in the era of large-scale digital "omic" data, the design of very efficient algorithms and sophisticated data structures is required for in silico analysis of multi-omic data. Thus, it is of importance to understand why and how an algorithm works, to be confident in its results, and how the data organization can influence the performance of algorithms. The goal of this chapter is to give an overview of fundamentals of algorithms and data structures design and evaluation to a non-informatician.

Key Points

- Algorithms time and space complexity analysis.
- Asymptotic notation.
- Notions of Data structures.
- Iterative algorithms.
- Recursive algorithms.

Introduction

Biology offers a huge amount and variety of data to be processed. Such data has to be stored, analyzed, compared, searched, classified, etcetera, providing new challenges to many fields of computer science. Among them, algorithmics and data structures design play a special role in the analysis of biological sequences, structures, and networks. Indeed, especially due to the flood of data coming from sequencing projects as well as from its down-stream analysis, the size of digital "omic" data to be studied requires the design of very efficient algorithms and sophisticated data structures. Moreover, biology has become, probably more than any other fundamental science, a great source of new algorithmic problems asking for accurate solutions. Nowadays, biologists more and more need to work with *in silico* data, and therefore it is important for them to understand why and how an algorithm works, in order to trust its results. The goal of this chapter is to give an overview of fundamentals of algorithms and data structures design and evaluation to a non-computer scientist.

Algorithms and Their Complexity

Computationally speaking, a *problem* is defined by an input/output relation: we are given an input, and we want to return as output a well defined solution which is a function of the input satisfying some property.

An *algorithm* is a computational procedure (described by means of an unambiguous sequence of instructions) that has to be executed in order to solve a computational problem. An algorithm solving a given problem is correct if it outputs the right result

for every possible input. The algorithm has to be described according to the entity which will execute it: if this is a computer, then the algorithm will have to be written in a programming language.

Example

Sorting Problem

- INPUT: A sequence S of n numbers $\langle a_1, a_2, \ldots, a_n \rangle$.
- OUTPUT: A permutation $< a_1', a_2', \ldots, a_n' >$ of S such that $a_1' \leq a_2' \leq \ldots \leq a_n'$.

Given a problem, there can be many algorithms that correctly solve it, but in general they will not all be equally efficient. The efficiency of an algorithm is a function of its input size.

A naive solution for the sorting problem would be to generate all possible permutations of S and, per each one of them, check whether this is sorted. This is clearly not efficient, as there is an exponential (in the input size n) number of such permutations and in the average case, as well as in the worst case, this algorithm would require a number $T(n)$ of elementary operations (such as write a value in a memory cell, comparing two values, swapping two values, etcetera) which is exponential in the input size n. In this case, since the worst case cannot be excluded, we say that the algorithm has an exponential *time complexity*. In computer science, exponential algorithms are considered *intractable*. An algorithm is, instead, *tractable*, if its time complexity function $T(n)$ is polynomial in the input size n. The *complexity of a problem* is that of the most efficient algorithm that solves it. Fortunately, the sorting problem is tractable, as there exist tractable solutions that we will describe later.

In order to evaluate the running time of an algorithm independently from the specific hardware on which it is executed, this is computed in terms of the amount of simple operations to which it is assigned an unitary cost or, however, a cost which is constant with respect to the input size. A constant running time is a negligible cost, as it does not grow when the input size does; moreover, a constant factor summed up with a higher degree polynomial in n is also negligible; furthermore, even a constant factor multiplying a higher polynomial is considered negligible in running time analysis. What counts is the growth factor with respect to the input size, i.e. the *asymptotic* complexity $T(n)$ as the input size n grows. In computational complexity theory, this is formalized using the *big-O* notation that excludes both coefficients and lower order terms: the asymptotic time complexity $T(n)$ of an algorithm is in $O(f(n))$ if there exist n_0 and $c > 0$ such that $T(n) \leq c\, f(n)$ for all $n \geq n_0$. For example, an algorithm that scans an input of size n a constant number of times, and then performs a constant number of some other operations, takes $O(n)$ time, and is said to have linear time complexity. An algorithm that takes linear time only in the worst case is also said to be in $O(n)$, because the big-O notation represents an upper bound. There is also an asymptotic complexity notation $\Omega(f(n))$ for the lower bound: $T(n) = \Omega(f(n))$ whenever $f(n) = O(T(n))$. A third notation $\Theta(f(n))$ denotes asymptotic equivalence: we write $T(n) = \Theta(f(n))$ if both $T(n) = O(f(n))$ and $f(n) = O(T(n))$ hold. For example, an algorithm that *always* performs a linear scan of the input, and not just in the worst case, has time complexity in $\Theta(n)$. Finally, an algorithm which needs to at least read, hence scan, the whole input of size n (and possibly also perform more costly tasks), has time complexity in $\Omega(n)$.

Time complexity is not the only cost parameter of an algorithm: *space complexity* is also relevant to evaluate its efficiency. For space complexity, computer scientists do not mean the size of the program describing an algorithm, but rather the data structures this needs to keep in memory during its execution. Like for time complexity, the concern is about how much memory the execution takes in the worst case and with respect to the input size. For the sorting problem, since the input array must be stored anyhow, a linear space complexity is also a lower bound. In this case, the challenge is whether or not the algorithm sorts *in place*, that is whether it requires additional linear space (e.g. another array as a working space) or just a constant number of constant-size variables. Also the exponential time complexity algorithm we described above has linear space complexity: at each step, it suffices to keep in memory only one permutation of S, as those previously attempted can be discarded. This observation offers an example of why, often, time complexity is of more concern than space complexity. The reason is not that space is less relevant than time, but rather that space complexity is in practice a lower bound of (and thus smaller than) time complexity: if an algorithm has to write and/or read a certain amount of data, then it forcely performs at least that amount of elementary steps (Cormen *et al.*, 2009; Jones and Pevzner, 2004; Mäkinen *et al.*, 2023). On the other hand, memory consumption is however a challenge when data is large (as this is often the case with the *omic sciences*) because when data does not fit in main memory read and write operations become much slower, thus affecting time too.

Iterative Algorithms

An *iterative algorithm* is an algorithm which repeats a same sequence of actions several times; the number of such times does not need to be known a priori, but it has to be finite. In programming languages, there are basically two kinds of iterative commands: the **for** command repeats the actions a number of times which is computed, or anyhow known, before the iterations begin; the **while** command, instead, performs the actions as long as a certain given condition is satisfied, and the number of times this will occur is not known a priori. What we call here an *action* is a command which can be, on its turn, again iterative. The cost of an iterative command is the cost of its actions multiplied by the number of iterations.

From now on, in this article we will describe an algorithm by means of the so-called *pseudocode*: an informal description of a real computer program, which is a mixture of natural language and keywords representing commands that are those typical of programming languages. To this purpose, before exhibiting an example of an iterative algorithm for the sorting problem, we introduce the syntax of a fundamental elementary command: the assignment "$x \leftarrow E$", whose effect is to set the value of an

expression E to the variable x, and whose time cost is constant, provided that computing the value of E, which can contain on its turn variables as well as calls of functions, is also constant. We will assume that the input sequence S of the sorting problem is given as an array: an array is a data structure of known fixed length that contains elements of the same type (in this case numbers). The i-th element of array S is denoted by S[i], and reading or writing S[i] takes constant time. Also swapping two values of the array takes constant time, and we will denote this as a single command in our pseudocode, even if in practice it will be implemented by a few operations that use a third temporary variable. What follows is the pseudocode of an algorithm that solves the sorting problem in polynomial time.

```
INSERTION-SORT(S,n)
 for i = 1 to n − 1 do
     j ← i
     while (j > 0 and S[j − 1] > S[j])
       swap S[j] and S[j − 1]
       j ← j − 1
     end while
   end for
```

INSERTION-SORT takes in input the array S and its size n. It works iteratively by inserting into the partially sorted S the elements one after the other. The array is indexed from 0 to $n-1$, and a **for** command performs actions for each i in the interval [1, $n-1$] so that at the end of iteration i, the left end of the array up to its i-th position is sorted. This is realized by means of another iterative command, nested into the first one, that uses a second index j that starts from i, compares S[j] (the new element) with its predecessor, and possibly swaps them so that S[j] moves down towards its right position; then j is decreased and the task is repeated until S[j] has reached its correct position; this inner iterative command is a **while** command because this task has to be performed as long as the predecessor of S[j] is larger than it.

INSERTION-SORT takes at least linear time (that is, its time complexity is in $\Omega(n)$) because all elements of S must be read, and indeed the **for** command is executed $\Theta(n)$ times: one per each array position from the second to the last. The invariant is that at the beginning of each such iteration, the array is sorted up to position S[$i-1$], and then the new value at S[i] is processed. Each iteration of the **for**, besides the constant time (hence negligible) assignment j ← i, executes the **while** command. This latter checks its condition (in constant time) and, if the newly read element S[j] is greater than, or equal to, S[$j-1$] (which is the largest of the so far sorted array), then it does nothing; else, it swaps S[j] and S[$j-1$], decreases j, checks again the condition, and possibly repeats these actions, as long as either S[j] finds its place after a smaller value, or S[j] turns out to be the smallest, and hence it becomes the new first element of S. Therefore, the actions of the **while** command are never executed if the array is already sorted. This is the best case for time complexity of INSERTION-SORT: linear in n. The worst case is, instead, when the input array is sorted in the reverse order: in this case, at each iteration i, the **while** command has to perform exactly i swaps to let S[j] move down to the first position. Therefore, in this case, iteration i of the **for** takes i steps, and there are $n-1$ such iterations for each $1 \le i \le n-1$. Hence, the worst case running time is

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2)$$

In this case we say that it has worst case *quadratic* time complexity. As for space complexity, INSERTION-SORT works within the input array plus a constant number of temporary variables, and hence it sorts *in place*.

The algorithm we just described is an example of iterative algorithm that realises a quite intuitive sorting strategy; indeed, often this algorithm is explained as the way we would sort playing cards in one hand by using the other hand to iteratively insert each new card in its correct position. Iteration is powerful enough to achieve, for our sorting problem, a polynomial time – although almost trivial – solution; the time complexity of INSERTION-SORT cannot however be proved to be optimal as the lower bound for the sorting problem is not n^2, but rather $n \cdot \log_2 n$ (result not proved here). In order to achieve $O(n \cdot \log_2 n)$ time complexity we need an even more powerful paradigm that we will introduce in next section.

Recursive Algorithms: Divide and Conquer

A *recursive algorithm* is an algorithm which, among its commands, recursively calls itself on smaller instances: it splits the main problem into subproblems, recursively solves them and combines their solutions to build up the solution of the original problem. Such a technique is called *divide and conquer*. There is a fascinating mathematical foundation, that goes back to the arithmetic of Peano, and even further back to induction theory, for the conditions that guarantee correctness of a recursive algorithm. We will omit details of this involved mathematical framework. Surprisingly enough, for a computer this apparently very complex paradigm, is easy to implement by means of a simple data structure (the *stack*). In order to show how powerful induction is, we will use again our Sorting Problem as an example. Namely, we describe here the recursive MERGE-SORT algorithm which achieves $\Theta(n \cdot \log_2 n)$ time complexity and is thus optimal. Basically, the algorithm MERGE-SORT splits the array into two halves, sorts them (by means of two recursive calls on as many sub-arrays of size $n/2$ each), and then merges the outcomes into a whole sorted array. The two recursive calls, on their turn, will recursively split again into subarrays of size $n/4$, and so on, until the base case (the already sorted sub-array of size 1) is reached.

```
MERGE-SORT(S,p,r)
  if p<r then
    q ← ⌊(p + r)/2⌋
    MERGE-SORT(S,p,q)
    MERGE-SORT(S,q + 1,r)
    MERGE(S,p,q,r)
  end if
```

The merging procedure will be implemented by the function MERGE (pseudocode not shown) which takes in input the array and the starting and ending positions of its portions that contain the two contiguous sub-arrays to be merged.

Recalling that the two half-arrays to be merged are sorted, MERGE uses two auxiliary arrays L and R in which it temporarily copies the sorted half-arrays in order not to overwrite them and two indices along them sliding from left to right, and, at each step: makes a comparison between the current element in L and the current element in R, writes in S the smallest, and increases the index of the corresponding array which contains it. This is done until the contents of both the auxiliary arrays have been entirely written into the result. The time complexity of MERGE is therefore just linear in the total size of the two arrays L and R as it exploits the fact that they were independently already sorted. Given the need of calling the algorithm on different array fragments, the input parameters of MERGESORT, besides S itself, will be the starting and ending position of the portion of array to be sorted. Therefore, the problem input size is $n = \text{r-p} + 1$, and the initial call will be MERGE-SORT(S, 0, $n - 1$). Then the index q which splits S in two halves is computed, and the two so found subarrays are sorted by means of as many recursive calls; the two resulting sorted arrays of size $n/2$ are then fused by MERGE into the final result. The correctness of the recursion follows from the fact that the recursive call is done on a half-long array, and from the termination condition "p<r": if this holds, then the recursion goes on; else (p=r) there is nothing to do as the array has length 1 and it is trivially already sorted. Notice, indeed, that if S is not empty, then p>r can never hold as q is computed such that $\text{p} \leq \text{q} < \text{r}$.

The time complexity T(n) of MERGE-SORT is defined by the following recurrence relation:

$$\text{T}(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2 \cdot T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

because, with an input of size n, MERGE-SORT calls itself twice on arrays of size n/2, and then calls MERGE which takes, as we showed above, Θ(n) time.

We now show by induction on n that $\text{T}(n) = \Theta(n \cdot \log_2 n)$. The base case is simple: if $n = 1$ then S is already sorted and correctly MERGE-SORT does nothing and ends in $\Theta(1)$ time. If $n > 1$, assuming that $\text{T}(n') = \Theta(n' \cdot \log_2 n')$ holds for $n' < n$, then we have that $\text{T}(n) = 2 \cdot n/2 \cdot \log_2 (n/2) + n = n (\log_2 n - \log_2 2 + 1)$, which is in $\Theta(n \cdot \log n)$. It follows that MERGE-SORT has optimal time complexity. As for space, the algorithm MERGE-SORT uses S itself and two auxiliary arrays whose sizes, if summed, are at most that of S, and hence it does **not** sort in place.

Data Structures

A *data structure* (DS) is a way to store and arrange data for efficient access and modification. There are no data structures that are suitable for every kind of problem. Some data structures have properties that make them perform well for specific problems, as well as some limitations that make them not practical for others. A data structure that supports a specific set of operations is efficient if the running time and the memory usage of operations are as small as possible, and whether or not a DS is a good choice for a specific task depends on what are the operations that are more likely to occur.

Data structures can be *static* or *dynamic*, *linear* or *non-linear*, *homogeneous* or not. A static data structure has a fixed size: its content can be changed, by adding or removing data inside it, but without modifying the memory allocated for it; they mainly store data and have fast access to them. Conversely, dynamic DS can change their size when operations are performed on them and are thus just as large as needed, but at the cost of generally more time consuming operations.

A data structure is linear if its elements are stored sequentially and each element is connected only to its predecessor and to its successor; and it is homogeneous if the elements it contains are of the same type.

Examples: Arrays and Trees

The simplest example of a homogeneous linear data structure is the *array*: that is the way of storing a sequence of same-type elements in a row. The order of the elements in the array is determined by their index. Thus, it supports *random access* since each element can be accessed directly in constant time by using its index. An array has fixed length (possibly resizable), hence it is a static data structure.

Other examples of linear data structures are *stacks* and *queues*. Unlike arrays, they are dynamic and offer restricted access to the data stored. In particular, as the term stack suggests, the elements of this data structure can be only piled up: only the element at the top can be directly accessed or removed, and a new element can only be added at the top of the stack (similarly to a stack of plates). The queue data structure works as a queue of people: the element at the front can be directly accessed or removed (items

can *only* be removed from the front), and a new element can only be inserted at the end. We say that the stack implements a *last-in first-out* (LIFO) rule, whereas the queue a *first-in first-out* (FIFO) policy. Any operation for adding/removing elements to/from a stack or a queue takes constant time; that is, it requires a fixed/limited amount of steps, independently from the size of the data structure. The simplest idea of stack usage is to reverse a string of characters. We have already mentioned another usage of a stack data structure that is for processing the calls of a recursive algorithm (such as merge-sort). Instead we can use a queue to schedule tasks in accordance with the order in which they are received.

A non-linear example of DS is a *tree*. A tree is composed of a finite set of *nodes* (its elements) connected by *edges*, which convey the relationships between pairs of nodes, without creating cycles. A tree is *rooted* when a single node is distinguished from the others; that node is the *root*. In a rooted tree, the *depth* of a node x is the number of edges in the simple path from the root to x, and a *level* of the tree comprises all the nodes at the same depth. If (y,z) is an edge of the simple path from the root to a node, then y is the *parent* of z, and z is a *child* of y. Nodes with the same parent are *siblings*. The root is the only node with no parent; and a node with no children is a *leaf*. The *height* of a node x is the number of edges on the longest simple path from the x to a leaf.

Rooted trees can be represented by linked data structures that connect the elements. For instance, consider a rooted *binary tree* (in which each parent has at most two children: *left child* and *right child*). Each node can be represented by an object having a key, i.e. an attribute that identifies the node, and pointers (possibly null) to the parent node and to each of its children. Such a representation also applies to *k-ary trees*, in which each parent has at most k children (denoted by $\text{child}_1,\ldots,\text{child}_k$). However, if the number of children cannot be bounded, a different representation must be devised. An ingenious scheme allows to represent the object node with the attribute key and three pointers: one to the parent node, another to the first child and the last to the next sibling. Rooted trees can also be represented in other ways (we show an example in the next section). The best representation scheme depends on the application.

Heap: a tree-based data structure

The *heap* is a data structure that we will use in the next section to show how the choice of a suitable data structure can influence the complexity of an algorithm. Indeed, using the heap leads to an optimal algorithm for the problem of sorting an array S of elements.

The heap DS is an array A whose elements can be viewed as nodes of a *nearly complete* binary tree T and an index *heap-size*, which keeps the number of heap elements stored in A. A binary tree is nearly complete if the number of nodes at any level (apart from the lowest) is the maximum possible, and the lowest level can be partially filled with nodes that are compactly placed from the left to a certain point. Thus, a leaf can only be at depth $\log_2 n$ or $\log_2 n$-1, where n is the number of tree nodes.

In the heap data structure, each node of the nearly complete binary tree T is mapped into an element of the array A: the root of T is the element A[*1*]; the node child of $A[i]$ corresponds to element $A[2i]$ if it is a left child, or to element $A[2i+1]$ if it is a right child. For instance, the element $A[2]$ (resp. $A[3]$) is the left (resp. right) child of the root.

An array A is a *max-heap* if its elements satisfy the following *max-heap property*: any non-root element $A[i]$ is not greater than its parent (element $A[\lfloor i/2 \rfloor]$). Therefore, the element $A[1]$ of a max-heap is the maximum among all the elements in the heap.

Replacing, for example, the element $A[i]$, for some $i <$ *heap-size*, with a smaller value could result in violating the max-heap property, as the new element might be smaller than its children. A recursive algorithm *heapify(A,i)* can restore the max-heap property after such violation has taken place for A[i], at a cost proportional to the height of the node $A[i]$. When the replaced element is the root, the cost is thus $O(\log_2 n)$ because the height of a heap is $O(\log_2 n)$.

Heapsort

The algorithm HEAPSORT uses the data structure max-heap to efficiently sort an array S. The strategy consists in iteratively selecting the maximum among the unsorted elements and placing it in order. The key idea behind the HEAPSORT is to build a max-heap data structure with the elements of S and maintain in the heap all unsorted elements exploiting, at each iteration, the max-heap properties to: (i) select in constant time the maximum element among those that are not sorted yet and removing it from the heap; (ii) restore the max-heap property of all unsorted elements by calling *heapify(S,1)*.

What follows is the pseudocode of HEAPSORT.

```
HEAPSORT(S)
  Convert S[1..n] into a max-heap
  for i = n to 2 do
    Swap S[1] and S[i]
    heap-size ← heap-size-1
    heapify(S,1)
  end for
```

The time complexity of HEAPSORT is given by the time complexity for converting S[1..n] into a max-heap, which is $O(n)$, plus the time to complete the for-loop, which is $O(n \cdot \log_2 n)$. In fact, each of the *n-1* iterations takes time $O(\log_2 n)$, cost given by *heapify*. The time complexity of HEAPSORT is thus optimal as it is $O(n \cdot \log_2 n)$ like that of MERGE-SORT. Moreover, HEAPSORT performs an *in place* sorting: it uses the array S plus only a constant number of other values outside S itself. Therefore, HEAPSORT combines the best features of the sorting algorithms described in the previous sections. This has been achieved thanks to the choice of a proper data structure.

Dictionaries

Algorithms manipulate data that can grow or reduce in size during their execution, and they require operations to be performed on them. Elements of such a dynamic set, that we call *dictionary*, can be represented as objects that store information and are identified by a *key*. An example of a dictionary is the set of distinct k-mers (substrings of fixed length k) that appear in a string s; each k-mer can be associated with the number of its occurrences in s and it is identified by a key that encodes the sequence of its characters.

- Operations such as INSERT to insert new elements, DELETE to remove elements, and SEARCH to test the belonging to the set, are the most common and required for a dictionary.
- A *hash table* is a data structure that efficiently implements a dictionary and its operations. In the worst case, the SEARCH operation can require $\Theta(n)$ time, but with reasonable assumptions the expected time for this operation is constant on the average.
- A *direct-address table* maps each key of the set to its corresponding object. Generalizing the element access on array, each position k of the table maps to the object having key k or to null (if k is not in the set of keys). In this way, any operation of INSERT, DELETE or SEARCH takes only constant time. However, direct addressing works well if the set of keys is reasonably small, but this is not often the case in practice. A hash table T uses then a *hash function* h to map a key to the position $h(k)$ of T where the object with key k is stored. Thus, the size of a hash table is typically much smaller than the range of all possible keys, and hence it saves space (and also search time) with respect to a direct-address table.
- A *perfect* hash function maps each key to a different position in T. Such a function can be defined if all the keys are known in advance, and in this case, searches are supported in constant time in the worst-case. Otherwise, we could have that two keys hash to the same position (*collision*). There exist effective techniques to resolve the conflicts given by collisions (two common methods are *chaining* and *open addressing*) and require constant time on the average for the basic dictionary operations.

Data Structures in Bioinformatics

Given the possibly huge size of data in bioinformatic applications, it is of primary importance to design memory efficient data structures that support efficient searching, storing, and manipulation of sequences, trees and graphs. In the following we describe some classical examples of data structures commonly used in several fields of computational biology.

In sequence analysis, *suffix arrays* and *suffix trees* are commonly used. A *suffix* of a string s starting at position i is the sequence of characters in s from i to the end of s. The suffix array of a string s is an array storing the starting positions of all the suffixes of s sorted by lexicographic order. The suffix tree of s is a rooted tree where edges are labeled with non-empty substrings of s, a non-leaf node has at least two children whose labels start with different characters, and any root-to-leaf path spells a different suffix of s. Suffix arrays and suffix trees are used, for instance, to find all (maximal) repeats in a string s (substrings that occur more than once in s), or to compute all-vs-all suffix-prefix overlaps for a set of strings (a fundamental task for genome assembly).

A natural extension to the notion of tree is that of a graph, where also cycles are allowed. A common graph data structure, often used to store DNA fragments, is the de Bruijin graph. A de Bruijin graph of order k is a graph whose nodes represent strings of fixed length k-1 and two nodes, u and v, are connected by an edge if there exists a k-mer that starts with the string in u and ends with string in v.

In phylogenetic analysis, a tree structure is generally used for displaying and studying evolutionary relationships among organisms. The leaves of the *phylogenetic tree* represent contemporary species and internal nodes represent hypothetical ancestor organisms from which speciation events where distinct lineages split. A Phylogenetic tree can be rooted or unrooted; the key difference is that in a rooted tree the basal node represents the inferred most recent common ancestor of the tree. A phylogeny can be represented by a binary tree, if it is fully resolved, or by a multifurcating tree, if some nodes have more than two children (polytomy). Another way of representing phylogenies is by using a graph, the *phylogenetic network*, which introduces hybrid nodes (nodes with two parents) that can represent reticulation events such as horizontal gene transfer or hybridization.

Closing Remarks

In this chapter we gave an overview of algorithms and their complexity, as well as of the complexity of a computational problem and how the latter should be stated. In particular we (i) described some fundamental paradigms in algorithms design: iteration, recursion, and the use of efficient data structures, and (ii) we showed the impact of data structures design.

References

Cormen, T.H., Leiserson, C.E., Rivest, R.L., *et al.*, 2009. Introduction to Algorithms, third ed. Boston, MA: MIT Press,.
Jones, N.C., Pevzner, P.A., 2004. An Introduction to Bioinformatics Algorithms. Boston, MA: MIT Press.
Mäkinen, V., Belazzougui, D., Cunial, F., 2023. Genome-Scale Algorithm Design: Biological Sequence Analysis in the era of High-Throughput Sequencing, 2nd Edition, Cambridge: Cambridge University Press.